

Портфолио бизнес-аналитика

Никита Яцинович

Senior Business Analyst | AI/ML и блокчейн-системы

Опыт в 9 доменах:

FinTech • Здравоохранение • E-commerce • GameFi • Страхование •
Недвижимость • ERP

Содержание

Проект 1: Платформа по переработке вторсырья с блокчейн

Проект 2: Text-to-SQL Analytics Platform



УВЕДОМЛЕНИЕ О КОНФИДЕНЦИАЛЬНОСТИ

Данное портфолио содержит обобщённые кейсы, основанные на моём профессиональном опыте. Вся информация о клиентах, бизнес-метрики, технические реализации и проприетарные детали были обезличены, изменены или абстрагированы в целях соблюдения конфиденциальности по условиям NDA.

Материалы демонстрируют методологию, подход и навыки решения задач, а не реальные производственные системы. Примеры API, SQL-запросов и диаграмм приведены исключительно в иллюстрационных целях и не отражают фактическую реализацию.

Проект 1: Платформа по переработке вторсырья с блокчейн

Клиент: Технологическая B2C компания | **Роль:** Middle Business Analyst |

Период: Сентябрь 2022 - Декабрь 2023 (16 месяцев)

Задача

Мобильное приложение, мотивирующее пользователей сдавать вторсырьё через автоматизированное устройство. После идентификации через QR-код,

устройство принимает упаковку, начисляет цифровое вознаграждение, которое можно использовать в системе бонусов.

Ключевые вызовы:

- Продукт полностью с нуля (нет ни MVP, ни specifications)
 - Интеграция с внешним партнёром-поставщиком устройств (документация на азиатском языке, отсутствие англоязычной версии — организовал процесс перевода и валидации через техническую команду партнёра)
 - Архитектурный выбор блокчейна и последующая интеграция
 - Real-time синхронизация статусов устройства (онлайн/оффлайн, заполненность)
 - Создание собственной токеномики
-

Моя работа

1. Архитектура системы

Спроектировал 5 основных модулей:

- **Управление пользователями:** регистрация, создание аккаунта, профиль
 - **Интеграция с устройством:** статусы, прием сырья, обновление в реальном времени
 - **Блокчейн-интеграция:** создание транзакций, начисление токенов, вывод
 - **Маркетплейс наград:** каталог наград, покупка за токены, доставка
 - **Веб-панель администратора:** управление бизнес-правилами и пользователями
-

2. Бизнес-процесс: Сдача вторсырья (BPMN)

Диаграмма 2.1: Процесс переработки через устройство

4. User Story с Acceptance Criteria (на основе BPMN)

US-001: Сдача вторсырья через устройство

Как пользователь приложения
Я хочу сдать вторсырьё в устройство
Чтобы получить баллы на свой аккаунт

Acceptance Criteria (краткая версия для портфолио):

AC1: Happy Path — Успешная сдача

1. Пользователь открывает приложение → генерируется QR-код (TTL x сек)
2. QR сканируется на устройстве → валидация через `POST /api/v1/objects/verify-account`
3. Backend проверяет: аккаунт активен + устройство доступно
4. Устройство открывает окно → пользователь загружает вторсырьё
5. Датчики считают объём → Backend рассчитывает баллы
6. Создаётся блокчейн-транзакция → ожидание подтверждения (max x сек)
7. Баланс обновляется → Push: "Вам начислено x баллов!"

💡 Более детальный flow см. в [Диаграмме 2.1 \(BPMN\)](#).

AC2: Устройство недоступно (full)

- Backend проверяет статус устройства перед сканированием
- Если `current_fill >= capacity`
 - Вернуть ошибку: HTTP 409 Conflict
 - Приложение показывает: "Устройство недоступно. Ближайшее в x м"

AC3: Превышен лимит сдачи вторсырья за день

- User сдал >x единиц вторсырья сегодня
- Backend возвращает ошибку: 429 Too Many Requests
- Приложение показывает: "Лимит x единиц вторсырья/день. Приходите завтра!"

🔌 5. API спецификация (компактная версия)



Примечание: Для краткости показаны только коды ошибок. В production документации каждый код содержит: структуру тела ответа, сообщение для пользователя и обработка на клиенте.

API 1: Создание сессии переработки

POST /api/v1/objects/submit-session

Request:

```
{
  "user_id": "uuid",
  "object_id": "устройство_001",
  "items": [
    {"category": "material_type", "quantity": 5}
  ]
}
```

Response 201:

```
{ "session_id": "uuid", "points_earned": 10.00, "blockchain_tx": "0x123...", "status": "pending" }
```

Errors:

400 INVALID_TYPE → "Неверный тип вторсырья"

409 Conflict → "Устройство недоступно. Ближайшее в 800 м"

429 RATE_LIMIT → "Превышен лимит x единиц вторсырья/день"

500 BLOCKCHAIN_ERROR → "Технический сбой"

✅ 6. Достижения и результаты:

Бизнес и продуктовые результаты:

- Uptime: 99%+ бесперебойной работы
- Успешность блокчейн-транзакций: 98%

- Достигнут break-even токеномики (burn rate = earn rate)
- Уровень фрода снижен с 5% до 0.5%
- Продукт внедрён в 10+ странах за 16 месяцев

Техническая реализация:

- **Мобильное приложение** (iOS + Android)
- **Блокчейн-интеграция:** провел сравнительный анализ х блокчейнов по критериям: стоимость, скорость транзакций, децентрализация, развитость и планы экосистемы.
Результат: Выбранная платформа обеспечила баланс между стоимостью (\$X per tx) и скоростью (x сек finality), что позволило достичь 98% успешности транзакций.
- **Интеграция с API** устройств стороннего поставщика
- **Админ панель** (управление бизнес-правилами, устройствами, пользователями)
- **50+ User Stories** с детальными Acceptance Criteria
- **25+ API endpoints** спроектировано и задокументировано
- **ERD с 10+ entities** (users, accounts, transactions, events, objects, etc.)
- **20+ BPMN-диаграмм** для всех процессов
- **Спецификация** интеграции с устройствами через Partner API

7. Сложности и решения

Проблема #1: Блокчейн — медленные подтверждения

Контекст: Иногда подтверждение транзакции занимало 2-3 минуты вместо 30 секунд

Решение:

- Увеличена комиссия (gas fees) на 20% для повышения приоритета
- Добавлена логика повторных попыток с экспоненциальной задержкой (до 3 попыток)
- Если все провалены → создаём транзакцию для ручной проверки

- Админ получает сообщение и может вручную перевести баллы
- Пользователь получает: "Баллы будут начислены в течение 1 часа"

Результат: Уровень успешных транзакций вырос с **85% до 98.5%**

Проблема #2: Попытки мошенничества

Контекст: Некоторые пытались сдавать одни единицы вторсырья несколько раз, перемещаясь между устройствами

Решение:

- Лимит: максимум x единиц вторсырья/день на пользователя
- Проверка геолокации: пользователь должен быть в радиусе x м от устройства
- Таймаут сессии: после сканирования QR есть x времени на сдачу
- Сканирование дублей: устройство отправляет штрих-код в систему
- Подозрительная активность → ручная проверка через админ-панель

Результат: Уровень фрода снизился с **5% до 0.5%**

Проект 2: Text-to-SQL Analytics Platform

Клиент: Интернациональная B2B SaaS компания

Период: Декабрь 2023 - настоящее время (1 год 11 месяцев)

Моя роль: Senior Business Analyst

Задача

Создать систему, которая преобразует вопросы на естественном языке в точные SQL-запросы для корпоративных данных.

Пример:

Пользователь спрашивает: *"Покажи закрытые дефектные задачи за прошлый месяц по направлениям"*

→ Система генерирует SQL и возвращает результат из хранилища данных (DWH).

Ключевые вызовы:

- Продукт полностью с нуля (нет ни MVP, ни спецификаций)
- Интеграция с несколькими SaaS-системами (Платформа для автоматической загрузки и синхронизации данных из разных источников в DWH, LLM-системы)
- Каждый клиент имеет свою терминологию ("bugs" vs "issues" vs "defects") - требовалась гибкая интерпретация
- LLM-only подходы "галлюцинируют" несуществующие таблицы и колонки (точность ~73%)
- Требование: $\geq 80\%$ точности SQL при времени ответа ≤ 10 секунд

Моя работа

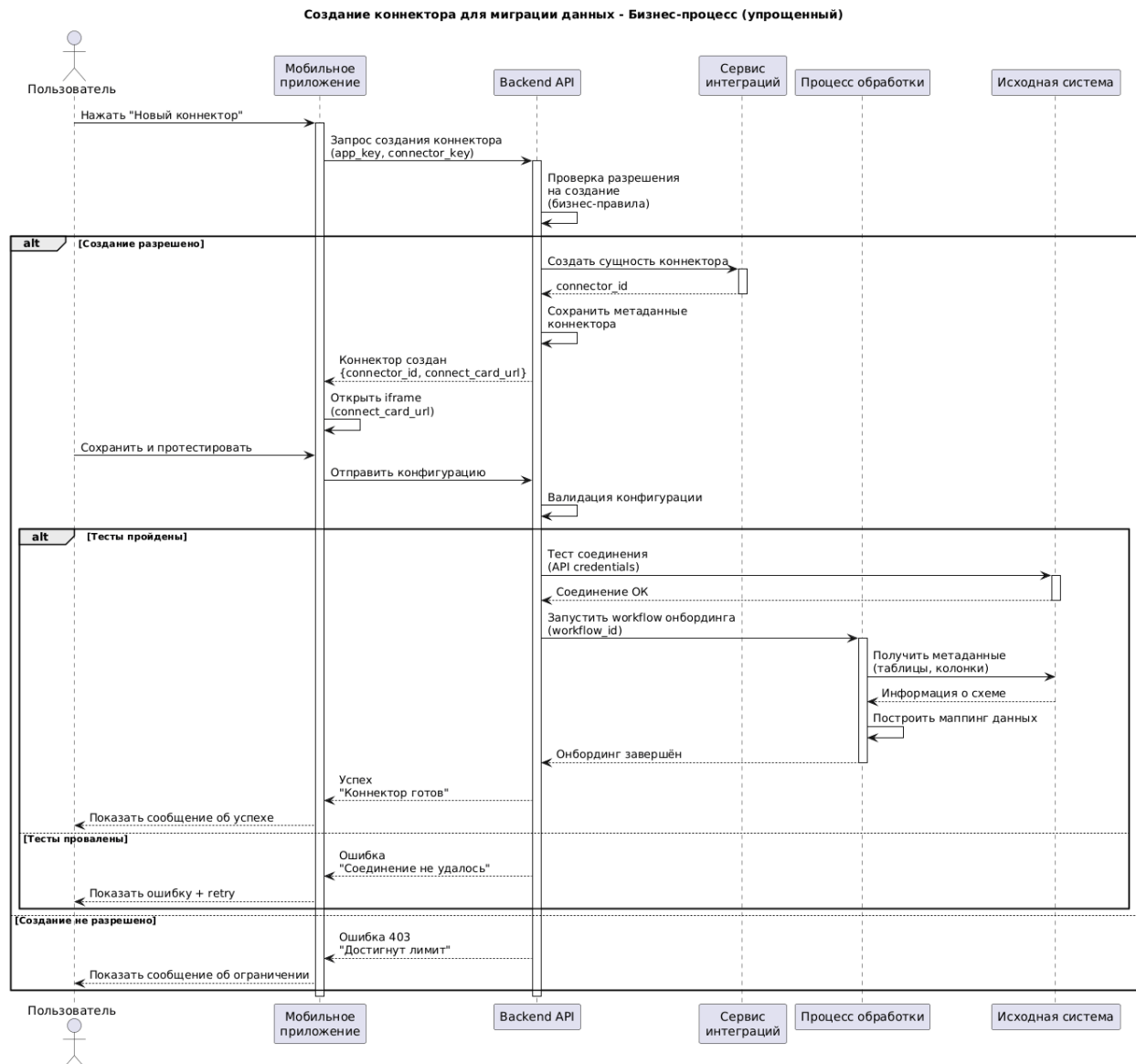
1. Архитектура системы

Спроектировал многоэтапный пайплайн обработки пользовательских запросов.

Основные компоненты:

- Модуль анализа запроса: определяет тип операции
- Модуль разрешения сущностей: использует гибридные методы поиска
- Модуль контекста: обрабатывает временные параметры и группировки
- Модуль метаданных: определяет связи между сущностями
- Генератор запросов: компилирует в SQL для аналитической СУБД

2. Бизнес-процесс в виде sequence-диаграммы: создание коннектора для миграции данных



Примечание: Показан упрощённый бизнес-процесс для демонстрации основных взаимодействий. Техническая реализация включает дополнительные технические шаги, детальную обработку ошибок и механизмы retry.



3. User Story с Acceptance Criteria: обработка вопроса пользователя по enterprise-данным

US-001: Обработка запроса на естественном языке (на примере вопроса)



Как пользователь

Я хочу задать вопрос в чате

Чтобы получить данные своей компании

Acceptance Criteria (краткая версия для портфолио):

AC1: Happy Path — Успешная обработка запроса

1. Пользователь вводит: "Покажи закрытые дефекты за прошлый месяц по группам".
2. Этап 1 определяет:
 - `intent: "filter"` (confidence: x)
Примечание: confidence может измеряться в диапазоне от 0 до 1.0
 - Параметры: `object_type` , `filter` , `time_range` , `grouping`
3. Этап 2 выполняет hybrid search:
 - "задачи" → `work_item` (confidence: x)
 - "закрытые" → semantic context → `status_group='Completed'` (confidence: x)
 - "группам" → `group.name` (confidence: x)
 - Все confidence ≥ y → автоматическое принятие
4. Этап 3 обрабатывает:
 - Time: "за прошлый месяц" → `yyyy-mm-dd` to `yyyy-mm-dd`
 - Filter: `status_group = 'Completed'`
 - Grouping: `group.name`
5. Этап 4 строит:
 - JOIN path: `work_item → group` (via `group_id`)
 - Валидация: loss-free (dup_ratio: x)
6. Этап 5 создаёт SQL-запрос в DWH - обобщённый пример для портфолио:

```
SELECT g.name, COUNT(w.id)
FROM work_items w LEFT JOIN groups g ON w.group_id = g.id
WHERE w.status_group = 'Completed' AND w.created_at BETWEEN '2024-09-01' AND '2024-09-30'
GROUP BY g.name
```

7. DWH возвращает результат.
8. Пользователь получает ответ за < 12 секунд.
9. Ответ содержит: агрегированные данные, конфигурацию визуализации (для диаграмм), исполненный SQL и общее время обработки.

AC2: Entity Resolution с низкой уверенностью (Clarification)

1. Пользователь вводит: *"Покажи статус по группам"*.
2. Этап 3 находит:
 - "статус" → Candidates:
 - `work_item.status` (confidence: x)
 - `group.status` (confidence: x)
 - Top confidence < y → триггер уточнения
3. Сервис 3 возвращает Response 200: Clarification needed:

```
{
  "status": "clarification_needed",
  "clarification_prompt": "Какой статус вас интересует: задач или направлений?",
  "options": [
    { "id": "opt_1", "label": "Статус задач (открыто/в работе/закрыто)", "maps_to": "task.status", "confidence": "x" },
    { "id": "opt_2", "label": "Статус направлений (активный/архивный)", "maps_to": "group.status", "confidence": "x" }
  ]
}
```

4. Пользователь выбирает: "Статус задач".
5. Система обновляет mapping: "статус" → `work_item.status` (confidence: 1.0, т.е. максимальная)
6. Пайплайн продолжает работу с resolved entity.

АС3: Валидация пути объединения (JOIN Path Validation)

1. Пользователь запрашивает данные из нескольких бизнес-сущностей.
2. Система ищет путь между объектами в графовой модели связей.
3. Путь проходит валидацию по критериям:
 - Допустимый уровень дублирования
 - Достаточное покрытие данных
 - Ограниченная длина пути
4. Если валидация не пройдена:
 - Возвращается ошибка 422: `"UNSAFE_JOIN_PATH"`
 - Предлагается альтернативный запрос
5. Если путь валиден:
 - JOIN добавляется в структуру запроса
 - Обработка продолжается

АС4: Обработка ошибок (Error Handling)

1. При ошибке backend (500):
 - Система логирует трассировку и контекст
 - Отправляет алерт в мониторинг-систему
 - Возвращает клиенту:

```
{ "status": "error", "error": { "code": "INTERNAL_ERROR", "message": "Внутренняя ошибка сервера", "trace_id": "sample-trace-id" }}
```

2. При таймауте хранилища данных (504):

- Выполняется до x повторных попыток с экспоненциальной задержкой
- При неудаче - вернуть пользователю сообщение: "Попробуйте позже"

✓ 4. Достижения и результаты

Технические достижения:

- **250+ User Stories** с детальными Acceptance Criteria и 200+ страниц документации
- **75+ API endpoints** спроектировано и задокументировано (gRPC и REST)
- **ERD с 8+ сущностями** (Business Glossary ↔ Warehouse schema) и 100+ таблиц в схеме БД
- **4-этапный пайплайн** с детерминированной обработкой
- **Confidence gates** на каждом этапе
- **Достигнута целевая точность SQL: >80%**
- **Латентность ответа:** 10-15 сек (target: <10 сек, 75% запросов укладываются)
- **Оптимизация LLM:** 80-85% сокращение операционных расходов на API
- **Графовая модель** с 200 сущностями
- Построена гибридная архитектура: правила + fallback на генеративную модель
- Десятки правил классификации семантических ролей данных
- 8+ интеграций с внешними enterprise-системами
- 25+ product events отслеживается в MixPanel
- 8+ conversion funnels для анализа воронок
- Соблюждён GDPR-комплаенс для европейского рынка
- ETL-процесс миграции 500K записей выполнен успешно

Процессные и командные результаты:

1. Командное взаимодействие и процессы

- Внедрил **pre-refinement сессии** с техлидом: сократил время груминга с **2 ч до 1 ч**. Через 3 месяца метрика 'stories rejected in sprint' снизилась с **18% до 7%**
- Организовал **5 воркшопов** по BPMN, API design, data modeling для cross-functional команды
- Инициировал создание единого **Business Glossary** для 3 команд (15 человек)

2. Стратегические решения и согласования

- Инициировал недельный **roadmap planning**, в рамках которого предложил trade-off: ранняя vs поздняя интеграция биллинга
 - Ранняя: +2 недели разработки, но чистая архитектура
 - Поздняя: быстрый MVP, но 3x tech debt при интеграции

Решение: Ранняя интеграция.
Результат: биллинг запущен сразу после завершения roadmap planning, что в дальнейшем привело к нулевым переработкам концепции биллинга.
- **Roadmap на 1 год** согласован и выполнен: **40+ функций из 45** без критических блокеров

5. Сложности и решения

Проблема #1: Терминологическая вариативность

Контекст: Разные клиенты используют разную терминологию:

- Клиент А: "bugs", "defects", "issues"
- Клиент Б: "tickets", "problems", "incidents"

Решение:

- Создал систему **семантических ролей**: обобщённые категории для консистентности данных
- Предоставление семантического описания таблиц и колонок для клиентов
- Разработал **бизнес-глоссарий с синонимами**:

```
{ "name": "closed_bugs", "synonyms": ["закрытые баги", "resolved issues", "done defects"], "maps_to": { "filter": {"column": "status_category", "value": "Done"} }}
```

- Внедрил **гибридный механизм поиска**, объединяющий семантические и текстовые сигналы

Результат: Точность резолвинга сущностей выросла до **91%** по внутренним сценариям

Проблема #2: LLM галлюцинации

Контекст: LLM-only подход генерировал несуществующие колонки или неправильные JOINS

Решение:

- Спроектировал **rules-first архитектуру**:
 - Детерминированные правила обрабатывают **80-90% случаев**
 - LLM вызывается только при confidence < x
- Разработал **confidence gate** на каждом этапе:
 - Этап 1: $\geq x$
 - Этап 3: $\geq x$
 - Этап 4: $\geq x$
- Внедрил **clarification loop**:
 - Низкая уверенность → вопрос пользователю
 - Ответ логируется → пополнение словаря синонимов

Результат: Сокращение некорректных SQL-выводов с **40% до <20%** по внутренним сценариям
